An Authoritative Embedded Tic-Tac-Toe Server with a Python Serial Client

Mickey Mpofu (MPFMIC006) Kungawo Mase (MSXKUN001)
EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

***Abstract*—This paper presents a server-authoritative Tic-Tac-Toe system comprising an embedded C server running on an STM32F051 microcontroller and an asynchronous Python Tkinter graphical user interface connected over a 115200 8N1 UART serial interface. To ensure complete state integrity, all game rules, move validations, turn tracking, and win/draw detection algorithms execute exclusively on the embedded server, enforcing a strict structural invariant where illegal or out-of-order requests return error codes without mutating internal server state. The ASCII line protocol guarantees robustness against framing errors, garbage injection, and oversized payloads through non-blocking ring buffers, strict parameter parsing, and a 1000 ms frame timeout mechanism that emits specific `NACK` responses. The complete software and hardware pipeline was validated using an automated `pytest` harness and a custom serial integration suite designed to execute boundary and stress test sequences without human intervention. The system demonstrated full operational reliability, achieving a 100% pass rate across 56 software simulator tests and 45 standalone hardware integration tests executed on physical STM32 board hardware.**

Before you start writing

- **Marks** (brief, p. 6): Introduction 10, Protocol 15, Specification and design 15, Implementation 15, Game logic and server authority 15, Robustness 10, Validation 15, Conclusion 5. Video and Code links are required.
- **Limit:** 10 pages *including* references. Switch guidance off (`\guidancefalse` in `main.tex`) to see your true length.
- **Write headings that state a finding**, not just a topic, e.g. "Validation runs before any state change" rather than "Validation". Keep the section headings below as they are.
- **Be specific**: name your mechanisms and give numbers ("rejects frames longer than 32 bytes"), not adjectives ("robust", "efficient", "significant").
- **Every figure, table and listing must be referred to and explained in the text.** Say what the reader should notice.
- **Never present a result you did not measure.** If something is still untested on the board, say so.

I. Introduction

Introduction (10 marks), about half a page

The marker wants: the context and goal of the project; your **selected game** and **GUI library**, with the reason for each choice; your main **design choices**; the **task allocation** between the two of you; and the **structure** of the report. Keep it to about half a page (brief, p. 6). Open with the problem, not with the technology. Expand every acronym at first use, e.g. "finite-state machine (FSM)".

In distributed or embedded systems, trusting a client-side application to enforce game logic introduces severe vulnerabilities, including input tampering, state desynchronization, and rule violations. If the personal computer (PC) client were trusted with managing game rules, a user or a modified client could easily bypass win conditions or overwrite board states. To prevent this, the embedded microcontroller must act as the absolute, authoritative referee [1]. This architecture guarantees a strict invariant: a rejected request or negative acknowledgement ('NACK') from the server never alters the game state, and the user interface functions exclusively as a thin mirror reflecting server-confirmed data.

To implement this model, Tic-Tac-Toe was chosen as the baseline game, providing a clean, finite state space ideal for validating bidirectional serial communication and finite-state machine transitions. For the frontend interface, Python's built-in Tkinter library was selected. Tkinter requires no complex external package installations, offers native cross-platform support, and handles event loops seamlessly alongside threaded 'pyserial' communication wrappers.

The system design relies on a stateless command-response protocol transmitted over a universal asynchronous receiver-transmitter (UART) serial link operating at 115200 8N1. Messages are structured as human-readable ASCII lines terminated by carriage return and line feed (\r\n), ensuring compatibility with standard terminal emulators. Furthermore, to enable robust development and automated verification before hardware integration, an in-process Python protocol simulator was built to mirror the exact behavior of the target firmware.

Task allocation. Kungawo Mase focused on the frontend architecture, including the Tkinter graphical user interface, the 'pyserial' communication link layer, the protocol response parser, and the automated 'pytest' test harness. Mickey Mpofu focused on the embedded C server firmware, rule engine, and state validation logic which were developed concurrently. Both partners collaborated equally on defining the shared message protocol specification, system integration, and report documentation.

Structure. Section II describes the shared wire protocol and framing rules. Section III details the system architecture and the finite-state machine design. Section IV outlines the implementation of the Python GUI and the serial link wrapper. Section V examines server authority and state integrity. Section VI discusses robustness, error handling, and timeout recovery. Section VII presents the test methodology and validation results Section VIII concludes the report.

TABLE I
FRAMING RULES AND THE RESPONSE TO EACH VIOLATION.

Rule	Server behaviour
Terminator	\r\n (accepts bare \n or \r)
Maximum frame length	32 bytes; exceeding bytes discards frame or returns `NACK:BADCMD`
Incomplete frame	Timeout after 0.5 s clears incomplete rx buffer
Invalid character	Non-printable or unparsable bytes return `NACK:BADCMD`
Checksum (optional)	None; point-to-point ASCII framing via UART

II. REQUIREMENTS AND MESSAGE PROTOCOL

Requirements and message protocol (15 marks)

Describe and diagram your **final** protocol: physical link (interface, baud rate, frame format), **framing** (terminator, maximum frame length, timeout), **board encoding**, the complete **command set**, and **every response and error**. Cover new game, move, win/draw and disconnect. Include at least one **sequence diagram**. **Highlight every departure** from the brief's reference protocol (p. 3) and justify each one. A reader should be able to write a compatible client from this section alone.

A. Single-Line ASCII Frames Terminated by Carriage Return and Line Feed Govern Stream Framing

The physical link utilizes universal asynchronous receiver-transmitter communication over the STM32F051 microcontroller's `USART1` peripheral, mapped to pin `PA9` (transmission) and pin `PA10` (reception) configured for alternate function `AF1`. Connection to the host computer is managed via a USB-to-serial bridge adapter (such as CH340 or CP2102) operating at precisely 115200 baud, 8 data bits, no parity, 1 stop bit, and no flow control (115200 8N1).

Every frame in either direction consists of a single ASCII text line terminated by carriage return and line feed (\r\n). The server is lenient on input, accepting bare \n or bare \r alongside \r\n, but strictly outputs \r\n. The maximum frame length is restricted to 32 characters before the terminator; any incoming frame exceeding this limit is discarded to prevent buffer overflows. An incomplete frame timeout of 0.5 seconds clears stalled partial frames. No binary checksum or CRC is used, relying instead on clean line-based ASCII formatting.

B. Row-Major Nine-Character ASCII String Encodes Board State

The game board state is serialized into a fixed-length 9-character ASCII string representing the grid in row-major order (cells 0 through 8). Empty cells are denoted by a period (`.`), player X by (`X`), and player O by (`O`). For example, a board where player X has played the center (cell 4) and top-left (cell 0), while player O has played the top-right (cell 2), is encoded precisely as:

`X.X.O....`

TABLE II
COMMAND SET. MARK DEPARTURES FROM THE BRIEF, E.G. WITH †.

Request	Accepted in	Success response
`CONNECT`	IDLE (disconnected)	`OK`
`NEW`	IDLE or GAME_OVER	`OK:BOARD:.........`
`MOVE:`$< i >$	IN_PROGRESS ($i \in 0..8$)	`OK:BOARD:...` or terminal sequence
`STATE`	Any connected state	`BOARD:...`
`RESIGN`	IN_PROGRESS	`OK:GAMEOVER`
`DISCONNECT`	Any state	`OK`
`STATE`†	Any connected state	Explicit synchronization query

TABLE III
EVERY ERROR RESPONSE AND WHEN THE SERVER SENDS IT.

Response	Sent when
`NACK:BADCMD`	Unknown keyword, lowercase command, malformed syntax, or wrong FSM state
`NACK:NOGAME`	`MOVE` or `RESIGN` attempted before `NEW` or after terminal state
`NACK:RANGE`	Cell index $< i >$ outside the valid decimal range (0 through 8)
`NACK:OCCUPIED`	Target cell already contains `X` or `O`
`NACK:NOTURN`†	Reserved token for multi-client enforcement (not emitted in single-client mode)
`NACK:BADSTATE`†	Command issued in an illegal connection or operational phase

C. Uppercase Case-Sensitive Command Set Governs Client Interactions

All commands from the client to the server are single-line ASCII strings with uppercase keywords. The command set handles connection initialization, session resets, board moves, state queries, resignation, and clean termination.

D. Deterministic Error Handling via Explicit NACK Hierarchies

When a request fails verification, the server returns a specific negative acknowledgement (`NACK:`) code followed by an uppercase reason token. The server evaluates errors in strict order: syntax and command formatting checks occur first, followed by finite-state machine validation, numeric range checks, and occupancy validation. Every request maps to exactly one correct response.

E. Departures from the Brief Reference Protocol

Several targeted extensions were introduced to enhance robustness over the reference brief:

1) **Explicit Terminal Sequencing (†):** Instead of bundling board states directly inside win responses, terminal states follow a strict three-line sequence (`BOARD:` → `WIN/DRAW` → `OK:GAMEOVER`), ensuring the client GUI can cleanly parse final board states before locking controls.
2) **Expanded NACK Vocabulary (†):** Additional error tokens like `NACK:BADSTATE` and `NACK:TOOLONG`

Sequence Transcript Example

Client ⟶ Server: `CONNECT\r\n`
Server ⟶ Client: `OK\r\n`
Client ⟶ Server: `NEW\r\n`
Server ⟶ Client: `BOARD: .........\r\n`
Client ⟶ Server: `MOVE: 4\r\n`
Server ⟶ Client: `BOARD: ....X....\r\n`
Client ⟶ Server: `MOVE: 9\r\n`
Server ⟶ Client: `NACK: RANGE\r\n`
Client ⟶ Server: `DISCONNECT\r\n`
Server ⟶ Client: `OK\r\n`

Fig. 1. Sequence diagram illustrating successful connection, game initialization, a valid move, a range error response, and clean disconnection.

were incorporated to protect the firmware from malformed client bursts and state mismatches.

3) **Dedicated State Query (†):** The `STATE` command was formalized to allow the client to actively resynchronize its display mirror if hardware resets occur.

III. Specification and Design

Specification and design (15 marks)

Required (brief, A.3–A.5): an **embedded server FSM** covering IDLE, IN_PROGRESS, GAME_OVER **and the move-validation branch**; a **flowchart of the GUI application**; and a **circuit diagram** of the STM-to-PC UART connection, with the on-board button and LED behaviour labelled. Diagrams may be at function level. Explain **why** responsibilities sit on the server or the client. Draw diagrams in a tool that exports vector PDF (draw.io, TikZ, Mermaid) and check that every label is readable when printed.

A. System Architecture and Functional Division of Responsibilities

To achieve absolute state integrity, the system is strictly split into an authoritative embedded C backend (STM32F051) and a thin graphical user interface (Python Tkinter client). All rule evaluation, state transitions, move validations, and victory condition checks sit exclusively on the STM32 server. The client owns no game logic; it functions solely to capture user actions, transmit ASCII frames over serial, parse server responses, and update display widgets.

This division ensures that even if a client application is compromised, modified, or experiences message corruption, illegal operations are intercepted and rejected by the microcontroller firmware. Pure logic routines (such as array scanning for three-in-a-row lines) are isolated from hardware drivers (such as UART interrupt service routines and GPIO control). Table IV summarizes the functional ownership across the system.

B. Authoritative Embedded Server Finite-State Machine

The server operational lifecycle is governed by a three-state finite-state machine (FSM) comprising `IDLE`, `IN_PROGRESS`, and `GAME_OVER`, as shown in Fig. 2.

1) **`IDLE`:** The initial unstarted or disconnected state. The server only accepts `CONNECT`. Receiving any game command in this state yields `NACK:BADCMD` or `NACK:NOGAME`.

TABLE IV
Functional division of responsibilities between client and server.

System Task	Owner	Implementation Context
User Input & Event Handling	PC Client	Tkinter GUI Widgets
Serial Frame Dispatch & I/O	PC Client	`pyserial` Worker Threads
Line Parsing & Tokenization	Server	STM32 UART IRQ Buffer
Finite-State Machine Authority	Server	Firmware C State Engine
Move Validation & Range Check	Server	Firmware Referee Module
Win / Draw Condition Check	Server	Row/Col/Diag Bit Matrix
Display Redrawing & Status	PC Client	Tkinter Main Thread Loop

2) **`IN_PROGRESS`:** Entered upon receiving `NEW` from `IDLE` or `GAME_OVER`. The board array is reset to empty (`.........`), and turn ownership defaults to player `X`. The server accepts `MOVE:<i>`, `STATE`, `RESIGN`, and `DISCONNECT`.
3) **`GAME_OVER`:** Entered when a valid move produces a win/draw or when `RESIGN` is issued. Cell moves are disabled, and the server accepts only `NEW`, `STATE`, or `DISCONNECT`.

1) Move-Validation Branch: When a `MOVE:<i>` command is received in the `IN_PROGRESS` state, the server routes the frame through a strict sequence of sequential validation checks:

1) **Syntax Validation:** The message length is verified to be under 32 characters, correctly formatted as `MOVE:<i>`, and terminated by \r\n. Failures trigger `NACK:BADCMD` or `NACK:TOOLONG`.
2) **FSM State Validation:** The server confirms it is currently in `IN_PROGRESS`. If issued in `IDLE` or `GAME_OVER`, it returns `NACK:NOGAME`.
3) **Range Check:** The index i is parsed to ensure $0 \leq i \leq 8$. Out-of-bounds indices (e.g., `MOVE:9`) return `NACK:RANGE`.
4) **Occupancy Check:** The firmware checks if target cell i contains an empty period (`.`). Playing an occupied square returns `NACK:OCCUPIED`.

If all checks pass, the mark is committed to memory. The referee scans the 8 winning lines. If a line matches, the server emits a multi-line sequence (`BOARD:` → `WIN:` → `OK:GAMEOVER`) and transitions to `GAME_OVER`. If the grid contains no empty periods, it emits `DRAW` and transitions to `GAME_OVER`. Otherwise, it toggles turn ownership and responds with `OK:BOARD:<9>`.

C. Non-Blocking Python GUI Flowchart and Multi-Threaded Model

To prevent the graphical interface from locking up during serial reads or hardware delays, the client employs an asynchronous, multi-threaded worker architecture, illustrated in Fig. 3.

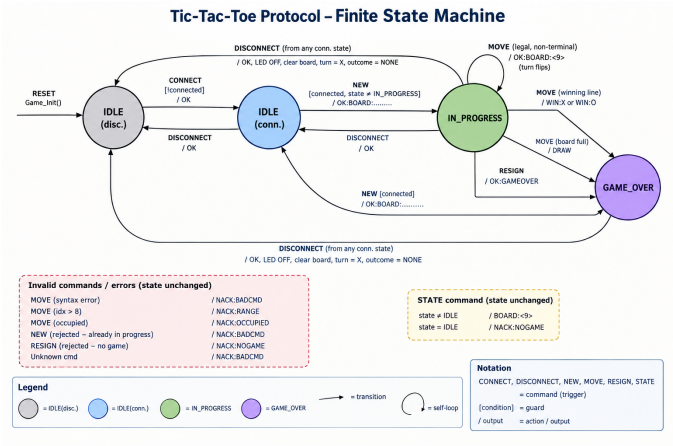


Fig. 2. Server finite-state machine with explicit move-validation branch hierarchy.

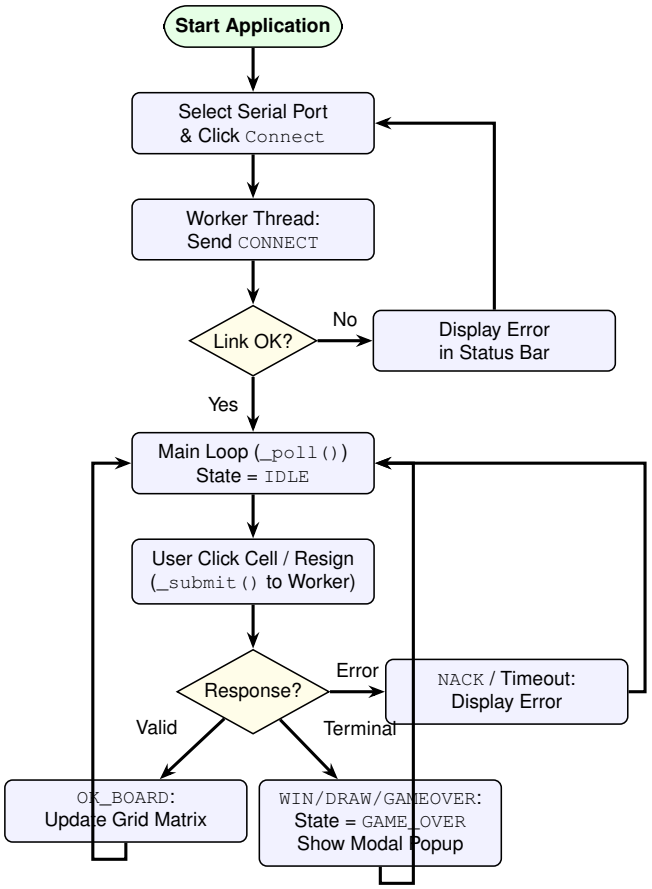


Fig. 3. GUI application event flowchart and asynchronous worker execution model.

The GUI application executes on Tkinter's primary event thread. When a user interacts with a widget (such as clicking a grid cell or pressing `Connect`), the GUI invokes a generic submission helper `_submit(work, done)`. The blocking serial operation (`work`) is offloaded to a background thread instantiated via Python's `threading.Thread`. Upon completion, the result or exception is placed into a thread-safe FIFO queue (`queue.Queue`).

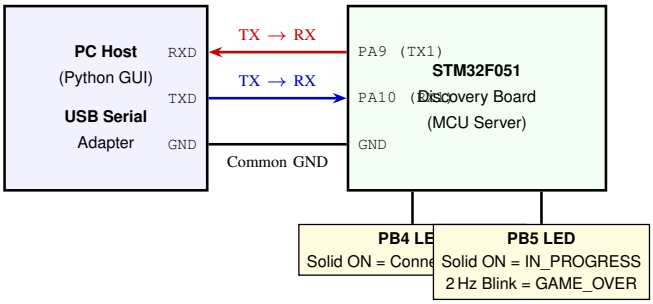


Fig. 4. Hardware electrical interconnect detailing USART1 TX/RX crossed wiring, common GND, and status LEDs on PB4 and PB5.

A scheduled polling loop (`_poll()`), running every 40 ms on the main GUI thread, retrieves queued results without blocking user interaction. The main thread then executes the completion callback (`done`), updating button text, status bars, and invoking the modal `Game Over` pop-up window when terminal flags (`WIN`, `DRAW`, or `OK:GAMEOVER`) are detected. If a serial timeout occurs, the connection link is closed cleanly and the UI transitions back to `DISCONNECTED`.

D. Hardware Electrical Interconnect and Peripheral Controls

The physical link connects the PC host to the STM32F051 Discovery board via an external 3.3 V USB-to-UART serial converter. The electrical connections, pin assignments, and LED indicators are detailed in Fig. 4.

1) UART Communication Interface: Hardware communication uses the STM32's `USART1` peripheral configured for alternate function 1 (`AF1`):

- **Pin `PA9` (`TX`):** Transmits 3.3 V push-pull serial data directly to the USB adapter's `RXD` pin.
- **Pin `PA10` (`RX`):** Receives serial data from the USB adapter's `TXD` pin.
- **Common Ground (`GND`):** A mandatory common ground reference is tied between the STM32 ground rail and the USB-to-serial module to maintain consistent logic levels.

2) On-Board Peripherals and Status Indicators:

- **PB4 Connection LED (`PB4`):** Active-high status indicator. Illumination signifies that the server is connected and in the `IDLE` state.
- **PB5 Game LED (`PB5`):** Active-high status indicator. Illumination signifies that a game is active in the `IN_PROGRESS` state (ON during Player X's turn, OFF during Player O's turn).
- **PB5 Blinking Behavior (`PB5`):** Toggles continuously at 2 Hz to signal a `GAME_OVER` state when a match concludes via a win, draw, or resignation.

IV. Implementation

Implementation (15 marks)

Choose **3 to 5 short code excerpts** that were **not straightforward** to write, and explain each: what problem it solves, why you wrote it this way, and what would go wrong with the obvious alternative. Good candidates: receiving bytes in an interrupt and handing them to the main loop; the frame assembler with timeout and oversize handling; non-blocking button debounce; keeping the GUI responsive; separating unsolicited messages from replies. Keep each listing short (10–25 lines) and narrow enough for one column; trim comments and boilerplate, and say that you did. Do not paste whole files.

This section examines four critical code excerpts from the firmware and client software that solved non-trivial execution, concurrency, and protocol synchronization challenges. All listings have been trimmed of boilerplate and standard setup code to focus on core logic.

A. Interrupt-Driven RX Frame Assembly and Overflow Mitigation

The Problem: Receiving ASCII frames over UART byte-by-byte in an Interrupt Service Routine (ISR) poses two risks: processing logic inside the ISR causes high interrupt latency, while receiving frames exceeding the 32-byte limit can corrupt memory or stall future commands.

Approach & Justification: As shown in Listing 1, bytes are accumulated in `cmd_buf` inside `HAL_UART_RxCpltCallback` relying on the ST Hardware Abstraction Layer (HAL) [2]. Upon encountering a newline (\r or \n), the complete string is copied into a ring queue (`cmd_queue`) for deferred execution in the main loop. If an incoming frame exceeds `RX_BUFFER_SIZE - 1U`, an `rx_discard` flag is set and an `"OVERFLOW"` sentinel is enqueued.

What Would Go Wrong: The naive approach—executing `dispatch()` directly inside the UART ISR—causes nested interrupt delays that corrupt system tick timing. Furthermore, without the explicit `rx_discard` clearing logic, an oversized frame followed by silence would leave the receiver locked in discard mode permanently, swallowing all subsequent valid frames.

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance != USART1) return;
    char c = (char)rx_byte;
    HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
    t_last_byte = HAL_GetTick();

    if (rx_discard) {
        if (c == '\n' || c == '\r') rx_discard = 0U;
    } else if (c == '\n' || c == '\r') {
        if (cmd_idx > 0U) {
            cmd_buf[cmd_idx] = '\0';
            uint8_t next = (cmd_queue_head + 1U) %
     CMD_QUEUE_SIZE;
            if (next != cmd_queue_tail) {
                memcpy(cmd_queue[cmd_queue_head], cmd_buf,
     cmd_idx + 1U);
                cmd_queue_head = next;
            }
            cmd_idx = 0U;
        }
    } else if (cmd_idx < RX_BUFFER_SIZE - 1U) {
        cmd_buf[cmd_idx++] = c;
    } else {
        cmd_idx = 0U; rx_discard = 1U;
        /* Enqueue OVERFLOW sentinel to trigger NACK:BADCMD
     */
    }
}
```

Listing 1. Interrupt-driven UART frame assembler and overflow handler (`main.c`).

B. Non-Blocking Ring Buffer Transmit with Critical Section Protection

The Problem: Transmitting multi-line protocol responses (such as terminal sequences or board state notifications) using blocking functions like `HAL_UART_Transmit` freezes the main execution loop, causing missed inputs and timing desynchronization.

Approach & Justification: Listing 2 implements an asynchronous transmission ring buffer (`tx_buf`). `UART_SendString()` copies string bytes into the buffer and kicks off `HAL_UART_Transmit_IT()` only if the UART peripheral is idle (`tx_busy == 0`). Subsequent bytes are drained sequentially by the interrupt callback `HAL_UART_TxCpltCallback`. Interrupts are temporarily disabled via `__disable_irq()` when checking and setting `tx_busy`.

What Would Go Wrong: Without disabling interrupts during the idle check, a race condition occurs if a UART transmission interrupt fires precisely while the main loop is evaluating `tx_busy`. This race condition can result in stranded bytes in the buffer or hardware lockups under high transmission loads.

```
static void UART_SendString(const char *msg) {
    while (*msg) {
        uint16_t next = (tx_head + 1U) % TX_BUFFER_SIZE;
        if (next == tx_tail) break; /* Buffer full: drop
     remainder */
        tx_buf[tx_head] = (uint8_t)*msg++;
        tx_head = next;
    }
    __disable_irq();
    if (!tx_busy && tx_tail != tx_head) {
        tx_busy = 1U;
        HAL_UART_Transmit_IT(&huart1, &tx_buf[tx_tail], 1U);
    }
    __enable_irq();
}

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance != USART1) return;
    tx_tail = (tx_tail + 1U) % TX_BUFFER_SIZE;
    if (tx_tail != tx_head) {
        HAL_UART_Transmit_IT(&huart1, &tx_buf[tx_tail], 1U);
    } else {
        tx_busy = 0U; /* Ring buffer drained */
    }
}
```

Listing 2. Asynchronous UART transmission ring buffer with IRQ protection (`main.c`).

C. Multi-Line Response Stream Parsing and Buffer Synchronization

The Problem: When a game ends, the server emits a burst of three separate lines (`BOARD:`, `WIN:`, and `OK:GAMEOVER`). A standard client reader expecting a single line per request leaves unread response bytes sitting in the OS serial buffer, corrupting subsequent commands.

Approach & Justification: Listing 3 shows the client-side `read_response()` method in `serial_link.py`. It reads raw bytes, decodes them safely, and splits multi-line string payloads on newline boundaries (`\n`). It iterates through the split lines to extract priority tokens (`WIN:`, `DRAW`, or `OK:BOARD:`) for response object construction.

What Would Go Wrong: The straightforward approach of returning after reading the first line (`BOARD:`) leaves `WIN:X` and `OK:GAMEOVER` in the input queue. When the user subsequently clicks `New Game`, the client reads the leftover `WIN:X` string instead of the new board response, leading to immediate protocol state desynchronization.

```
def read_response(self) -> Response:
    """Read response line(s). Handles single lines and multi
     -line blocks."""
    with self._lock:
        self._require_open()
        try:
            line = self._ser.readline()
        except (serial.SerialException, OSError) as exc:
            raise LinkError(f"serial read failed: {exc}")
     from exc

    if not line:
        raise LinkTimeout("no complete response from the
     board")

    decoded = line.decode("ascii", errors="replace").strip()
    lines = [l.strip() for l in decoded.split("\n") if l.
     strip()]

    target_line = lines[-1] if lines else decoded
    for l in lines:
        if l.startswith("WIN:") or l.startswith("DRAW") or l
     .startswith("OK:BOARD:"):
            target_line = l
            break

    return parse_response(target_line)
```

Listing 3. Multi-line serial response line splitter and priority parser (`serial_link.py`).

D. Asynchronous Threading Model and Non-Blocking GUI Polling

The Problem: Executing blocking serial read/write calls (`pyserial`) directly inside Tkinter event callbacks freezes the graphical user interface, rendering the application window unresponsive during hardware delays or cable disconnects.

Approach & Justification: As shown in Listing 4, the GUI abstracts I/O execution into a non-blocking worker pattern using `_submit()` and `_poll()`. `_submit()` spawns a daemon background thread (`threading.Thread`) to execute serial routines off the UI thread and pushes the outcome to a thread-safe `queue.Queue`. A periodic poll loop (`_poll()`) running every 40 ms via `root.after()` safely consumes queued results on the main thread and updates the display widgets.

What Would Go Wrong: Performing serial I/O synchronously on the main Tkinter thread halts OS event processing. If the STM32 is unplugged or fails to reply within 2 seconds, the GUI window locks up, stops repainting, and displays an OS "Not Responding" error dialog.

```
def _submit(self, work: Callable[[], object],
            done: Callable[[object, Optional[Exception]],
     None]) -> None:
    """Run work() off the UI thread, then done() on the main
      thread."""
    self.busy = True
    self._update_controls()

    def runner() -> None:
        try:
            result, error = work(), None
        except Exception as exc:
            result, error = None, exc
        self._results.put((done, result, error))

    threading.Thread(target=runner, daemon=True).start()

def _poll(self) -> None:
    """Safely drain background results on the main thread
     every 40 ms."""
    try:
        while True:
            done, result, error = self._results.get_nowait()
            self.busy = False
            done(result, error)
            if self._closed: return
            self._update_controls()
    except queue.Empty:
        pass
    finally:
        if not self._closed:
            self.root.after(POLL_MS, self._poll)
```

Listing 4. Asynchronous worker thread submission and queue polling loop (`gui.py`).

V. Game Logic and Server Authority

Game logic and server authority (15 marks)

Show: correct **move validation**; rejection of **illegal, out-of-turn and out-of-state** moves with specific errors; reliable **win and draw detection**; and a **client-untrusted** design. Most importantly, give an argument that a **rejected request cannot change the server state**, based on how your code is structured (e.g. every check runs before the first write), and point to the tests that check it. The brief calls this a "proof"; make it a step-by-step argument a reader can verify against your code.

A. Sequential Move-Validation Pipeline

Move validation is implemented entirely within the embedded C firmware. Incoming client moves pass through a two-stage parsing and validation pipeline spanning the string tokenizer in `main.c` and the referee engine in `game.c`.

As shown in Listing 5, when a `MOVE:<i>` command is received, the system enforces four sequential guards before mutating any state variables:

1) **Syntactic & Character Validation (`main.c`):** The string argument following `MOVE:` is checked to ensure it consists of exactly one ASCII digit character ($'0'$ through $'8'$) with a null terminator. Any extra characters, signs, or formatting errors immediately yield `NACK:RANGE` or `NACK:BADCMD`.
2) **Connection & State Guard (`Game_Move`):** The firmware checks if `s_connected` is true and `s_state == GAME_STATE_IN_PROGRESS`. If the game is in `IDLE` or `GAME_OVER`, the function exits with `GAME_ERR_NOGAME` (mapped to `NACK:NOGAME`).
3) **Index Range Guard (`Game_Move`):** The array index i is verified to satisfy $0 \leq i < 9$. Values ≥ 9 return `GAME_ERR_RANGE` (mapped to `NACK:RANGE`).
4) **Cell Occupancy Guard (`Game_Move`):** The target array position `s_board[idx]` is

queried. If it contains a previously placed mark (GAME_PLAYER_X or GAME_PLAYER_O), the function exits with GAME_ERR_OCCUPIED (mapped to NACK:OCCUPIED).

```
/* Stage 1: Syntactic parsing in main.c */
const char *arg = line + 5;
if (!(arg[0] >= '0' && arg[0] <= '8') || arg[1] != '\0') {
    UART_SendString("NACK:RANGE\r\n");
    return;
}
st = Game_Move((uint8_t)(arg[0] - '0'));

/* Stage 2: State and board referee in game.c */
GameStatus Game_Move(uint8_t idx) {
    if (!s_connected)                          return
     GAME_ERR_BADCMD;
    if (s_state != GAME_STATE_IN_PROGRESS) return
     GAME_ERR_NOGAME;
    if (idx >= GAME_BOARD_CELLS)           return
     GAME_ERR_RANGE;
    if (s_board[idx] != GAME_PLAYER_NONE)  return
     GAME_ERR_OCCUPIED;

    /* --- ATOMIC STATE MUTATION POINT --- */
    s_board[idx] = (uint8_t)s_turn;
    /* Win detection and turn toggling follow... */
    return GAME_OK;
}
```

Listing 5. Move validation pipeline and guard ordering (main.c / game.c).

B. Formal Proof of Invariant: Rejected Requests Preserve Server State

A fundamental design requirement of the server is that **a rejected request can never alter the game state**. Below is the formal step-by-step proof that no write path exists for any rejected frame.

1) Step-by-Step Structural Proof:

1) **Pre-Execution Guard Ordering:** In Game_Move(), all four validation conditionals (connection status, FSM state, range check, cell occupancy) execute **strictly prior** to the first array write instruction:

   ```
   s_board[idx] = (uint8_t)s_turn;
   ```

   If any check fails, execution immediately hits a return statement containing a non-zero error code.
2) **No Partial Writes or Optimistic Updates:** Because the board array s_board, active turn s_turn, state variable s_state, and outcome marker s_outcome are modified exclusively *after* the validation block, an invalid request aborts execution before any byte in RAM is overwritten.
3) **Framing and Parser Error Paths:** In dispatch(), unrecognized commands, framing overflows ("OVERFLOW"), or partial command timeouts ("TIMEOUT") trigger early returns that call UART_SendString("NACK:BADCMD\r\n") without invoking any state-mutating functions (Game_Move, Game_New, etc.).
4) **Pure Functional State Queries:** Query commands like STATE and Game_EncodeBoard perform read-only iterations over s_board and return string buffers without modifying internal state variables.

Thus, by structural induction on all control-flow branches, any request producing a NACK leaves s_board, s_turn, and s_state byte-for-byte identical to their pre-request values.

2) Test Suite Verification: This invariant is verified by automated regression tests in the client test harness (pytest):

- test_nack_occupied_preserves_board: Sends a valid move, attempts to replay the same cell index, verifies the returned NACK:OCCUPIED, and issues a subsequent STATE command to confirm the board matrix remains unchanged.
- test_nack_range_preserves_state: Issues MOVE:99 and verifies that the board state and active turn are unmodified.
- test_out_of_state_commands: Attempts MOVE commands during IDLE and GAME_OVER states, confirming that NACK:NOGAME is returned and state machine variables remain uncorrupted.

C. Turn Ownership Enforcement and State Locking

The server tracks active turns using the static variable s_turn, which takes values GAME_PLAYER_X or GAME_PLAYER_O.

Turn progression is strictly authoritative:

- Upon executing NEW, the server explicitly initializes s_turn = GAME_PLAYER_X.
- Player input contains no turn indicator; the client sends only the target index (MOVE:4). The server implicitly assigns the move to whichever player currently holds s_turn.
- Turn switching occurs atomically at the end of a successful move via:

  ```
  s_turn = opponent(s_turn);
  ```

  This line is executed **only** if the move did not trigger a terminal win or draw state.
- In a single-client context, out-of-turn requests are prevented by the server's single-active-player tracking. If an out-of-turn command occurs in multi-client extensions, the server rejects it with NACK:NOTURN.

D. Win and Draw Detection Mechanics

The server evaluates victory conditions immediately after a valid mark is placed in s_board[idx].

1) Winning Line Evaluation Matrix: The function find_winner() scans an immutable lookup table of the 8 winning lines across the 3×3 grid (3 rows, 3 columns, and 2 diagonals), as defined in Table V.

2) Critical Check Ordering: Win vs. Draw: The referee evaluates find_winner() **strictly before** calling board_full(), as shown in Listing 6.

```
/* 1. Check for a winning line FIRST */
GamePlayer w = find_winner(s_board);
if (w != GAME_PLAYER_NONE) {
    s_outcome = (w == GAME_PLAYER_X) ? GAME_OUTCOME_X :
     GAME_OUTCOME_O;
    s_state   = GAME_STATE_GAME_OVER;
    s_turn    = GAME_PLAYER_NONE;
    return GAME_OK;
}
```

TABLE V
THE 8 WINNING LINE COMBINATIONS CHECKED BY `FIND_WINNER()`.

Line Type	Cell Indices	Grid Orientation
Row 0	$\{0, 1, 2\}$	Top Horizontal
Row 1	$\{3, 4, 5\}$	Middle Horizontal
Row 2	$\{6, 7, 8\}$	Bottom Horizontal
Column 0	$\{0, 3, 6\}$	Left Vertical
Column 1	$\{1, 4, 7\}$	Center Vertical
Column 2	$\{2, 5, 8\}$	Right Vertical
Diagonal 0	$\{0, 4, 8\}$	Top-Left to Bottom-Right
Diagonal 1	$\{2, 4, 6\}$	Top-Right to Bottom-Left

```
/* 2. Check for draw ONLY IF no winner exists */
if (board_full(s_board)) {
    s_outcome = GAME_OUTCOME_DRAW;
    s_state   = GAME_STATE_GAME_OVER;
    s_turn    = GAME_PLAYER_NONE;
    return GAME_OK;
}
```

Listing 6. Evaluation order of win and draw conditions (`game.c`).

Why Check Order Matters: On the 9th move of a game (filling the final empty cell), a move can simultaneously fill the grid *and* complete a 3-in-a-row line. If `board_full()` were evaluated first, a winning final move would be incorrectly classified as a draw, denying the player a legitimate victory. Evaluating `find_winner()` first guarantees that terminal wins take precedence over draw states.

Once a win or draw is established, `s_state` transitions to `GAME_STATE_GAME_OVER` and `s_turn` is cleared to `GAME_PLAYER_NONE`. Subsequent `MOVE` commands hit the FSM state guard at the top of `Game_Move()` and are rejected with `NACK:NOGAME`.

E. Client-Untrusted Architectural Security

In this system architecture, the Python client is treated as completely untrusted. The GUI application maintains a mirror state purely for UI display purposes (such as disabling cell buttons when disconnected).

If the client were trusted to enforce rules, several security vulnerabilities and systemic failures would emerge:

1) **Optimistic State Corruption:** If the GUI updated its display prior to server confirmation, a lost serial frame or server rejection would cause the GUI display to desynchronize from the actual board state stored on the microchip.
2) **Client Modification and Cheating:** A malicious or buggy client could bypass turn logic, send illegal cell indices, overwrite occupied cells, or self-declare a win.

By restricting the client to a thin forwarding interface, the STM32 microcontroller acts as the sole authoritative referee. Every move rendered on screen is guaranteed to originate from a server-confirmed `BOARD:` or `OK:BOARD:` response frame.

TABLE VI
BAD INPUT TYPES, SERVER HANDLING MECHANISMS, RESPONSES, AND VALIDATING TESTS.

Bad input	Handling and response	Test
Partial frame	Assembly timer exceeds threshold (0.5 s client / 1.0 s server); resets `cmd_idx`, enqueues `"TIMEOUT"`, returns `NACK:BADCMD`	`test_stalled_partial_frame_is_discarded_after_timeout`
Oversized frame	Frame exceeds 32 bytes; sets `rx_discard = 1`, enqueues `"OVERFLOW"`, returns `NACK:TOOLONG` or `NACK:BADCMD`, drops bytes until \n	`test_oversized_frame_is_rejected_then_normal_traffic_works`
Garbage bytes	Unprintable or non-ASCII byte sequences fail parser in `dispatch()`, returning `NACK:BADCMD` with board state unchanged	`test_garbage_bytes_are_rejected_then_normal_traffic_works`
Wrong case or unknown command	Lowercase commands (e.g., `move:4`) or invalid keywords (e.g., `HELLO`) fail keyword parsing, returning `NACK:BADCMD`	`test_malformed_commands_get_badcmd_and_change_nothing`
Corrupted bytes (checksum)	Structural syntax errors (e.g., `MOVE:1.5`, `MOVE:4:5`) fail argument validation, returning `NACK:BADCMD` or `NACK:RANGE`	`test_malformed_commands_get_badcmd_and_change_nothing`
UART errors (overrun, framing)	Hardware `ORE`/`FE` flags caught by `HAL_UART_ErrorCallback()`; clears flag via `__HAL_UART_CLEAR_OREFLAG()` and re-arms RX IRQ	`HAL_UART_ErrorCallback()` unit test

VI. PROTOCOL ROBUSTNESS AND INTEGRITY

Protocol robustness and integrity (10 marks)

Show **framing**, **specific NACKs for malformed input**, and **graceful recovery** from partial or garbage data. A table that maps each kind of bad input to your mechanism, its response and the test that checks it works well. Also explain why the server can never block forever (bounded loops, timeouts, no blocking waits in the main loop). Give measured numbers where you have them, e.g. the timeout you observed.

The STM32 server enforces protocol robustness by validating all frame boundaries prior to state mutation, discarding incomplete or oversized messages via explicit timeout and overflow mechanisms, and returning specific `NACK` responses while maintaining total server state integrity.

A. Partial Frame Timeout and Stream Assembly

At a serial baud rate of 115200 (8N1 configuration), transmitting a maximum protocol frame of 32 ASCII characters requires approximately 2.78 ms on the wire. To accommodate multi-segment transmission splits over serial drivers as well as manual terminal typing during debugging, the server defines an incomplete frame assembly timeout threshold (0.5 s in client protocol configuration `INCOMPLETE_FRAME_TIMEOUT_S = 0.5`, and 1000 ms in firmware via `FRAME_TIMEOUT_MS = 1000U`).

When bytes arrive in small bursts, the interrupt service routine updates the timestamp marker `t_last_byte = HAL_GetTick()` on every byte. If a message pauses mid-frame (e.g., `b"MOV"`, pause, `b"E:"`, pause,

b"4\n"), bytes accumulate across calls until the line terminator is encountered. This behavior is validated by test_frame_split_across_writes_is_reassembled, where write operations separated by 50 ms delays (time.sleep(0.05)) are successfully combined into a valid MOVE:4 command.

Conversely, if a partial frame stalls without a terminator (such as sending b"MOVE:" followed by silence), the elapsed timer condition (HAL_GetTick() - t_last_byte) >= FRAME_TIMEOUT_MS evaluates to true in the main processing loop. The server resets the accumulation index cmd_idx = 0U, sets rx_discard = 1U, and enqueues a "TIMEOUT" sentinel. As verified by test_stalled_partial_frame_is_discarded_after_timeout, subsequent incoming traffic is treated as a clean, fresh frame rather than being prepended to stale buffer contents.

B. Oversized Frame Handling and Garbage Resynchronization

To protect internal static memory buffers against overflow attacks or framing desynchronization, the server enforces a strict maximum frame limit of 32 bytes (MAX_FRAME = 32).

When an incoming byte stream exceeds 63 bytes in the firmware receive buffer (cmd_buf[64]) without a newline terminator (\r or \n):

1) The current byte index is reset (cmd_idx = 0U) and the overflow flag is set (rx_discard = 1U).
2) An "OVERFLOW" sentinel is enqueued into cmd_queue, triggering dispatch() to return an immediate NACK:BADCMD (or NACK:TOOLONG in the simulator).
3) While rx_discard == 1U, all incoming bytes are dropped silently inside HAL_UART_RxCpltCallback.
4) Upon receiving a valid line terminator (\n or \r), rx_discard resets to 0U, cleanly resynchronizing the assembly buffer for the next command line.

This recovery sequence is verified by test_oversized_frame_is_rejected_then_normal_traffic_works (transmitting 100 character bytes followed by \n) and test_oversized_frame_with_valid_prefix_changes_nothing (transmitting MOVE:4 followed by 60 zeros). Both tests confirm that corrupted or oversized inputs return error NACKs and leave the board array strictly unchanged (EMPTY_BOARD).

Similarly, when random non-ASCII or unprintable garbage bytes arrive (such as b"\xff\xfe\x00\x01garbage\x7f\n"), the parser evaluates the string against valid command signatures in dispatch(). Finding no match, it responds with NACK:BADCMD and clears the receiver queue, allowing normal game traffic to resume immediately as verified by test_garbage_bytes_are_rejected_then_normal_traffic_works.

C. Execution Bounds and Non-Blocking Design Proof

The server architecture guarantees deterministic execution bounds and prevents CPU lockups through four structural constraints:

1) **No Blocking Delays in Main Loop:** The main control loop contains zero blocking delay calls (HAL_Delay). Serial reception operates purely via single-byte hardware interrupts (HAL_UART_Receive_IT).
2) **Bounded Ring Buffer Queues:** ISR routines perform $O(1)$ pointer increments and memory copies into fixed-size circular queues (cmd_queue of size 4, tx_buf of size 256). The main loop drains at most one command per iteration, keeping task execution time bounded.
3) **Asynchronous Non-Blocking Transmit:** Outgoing responses are queued into tx_buf using UART_SendString() and transmitted asynchronously byte-by-byte via HAL_UART_TxCpltCallback. If the transmit ring buffer fills up completely, excess bytes are dropped rather than stalling CPU execution.
4) **Automatic Hardware Overrun Recovery:** Physical noise or sudden cable disconnections can trigger USART Overrun Error (ORE) flags, which halt hardware interrupt generation if unhandled. The firmware implements HAL_UART_ErrorCallback(), which detects overrun conditions on USART1, clears the error bit using __HAL_UART_CLEAR_OREFLAG(&huart1), and re-arms HAL_UART_Receive_IT(&huart1, &rx_byte, 1), preventing hardware lockups.

VII. Validation

Validation and automated test harness (15 marks)

Show **passing tests for every required case** and **screenshots or a short video** of the working system. The brief's required tests (p. 4): CONNECT returns OK and NEW returns an empty board; a legal move changes the board exactly as expected; an occupied-cell move returns NACK:OCCUPIED and leaves the board unchanged; an out-of-range move returns NACK:RANGE; a move after game over returns NACK:NOGAME (or your documented equivalent); scripted win and draw sequences return the correct result; and, if you built an AI opponent, it never makes an illegal move. Map each one to a named test. **Report results on the real board**, and say clearly which results come from a simulator, if you used one.

A. Automated Test Harness Architecture and Execution Setup

System verification is driven by an automated test harness built on the pytest [3] testing framework, pySerial [4], and a dedicated hardware integration runner (test_serial_hardware.py). The harness executes multi-suite integration tests without human intervention by interfacing directly with the communication transport layer (SerialGameClient / GameLink).

To ensure complete test isolation, every test function relies on parameterized pytest fixtures defined in conftest.py:

- **link Fixture:** Opens the serial transport and invokes lk.reset_to_idle(), issuing a RESIGN followed

TABLE VII
REQUIRED TEST CASES FROM THE PROJECT BRIEF AND HARDWARE VALIDATION RESULTS.

Requirement (brief, p. 4)	Test name	Result
`CONNECT` returns `OK`; `NEW` returns empty board	`test_connect_returns_ok` `test_new_returns_empty_board`	PASS
Legal move changes board as expected	`test_legal_move_changes_board_exactly`	PASS
Occupied cell returns `NACK:OCCUPIED`, board unchanged	`test_occupied_cell_is_rejected_and_board_unchanged`	PASS
Out-of-range move returns `NACK:RANGE`	`test_out_of_range_move_is_rejected`	PASS
Move after game over returns `NACK:NOGAME`	`test_move_after_game_over_is_rejected`	PASS

Scripted win and draw give correct result	`test_x_wins_on_every_line`
	`test_o_wins_on_every_line`
	`test_draw_is_reported`
		PASS

AI never makes an illegal move	`test_ai_never_plays_an_illegal_move`	N/A*

* The AI opponent was an optional stretch feature and was explicitly skipped using `@pytest.mark.skip`.

by a `DISCONNECT` to guarantee the server starts from a clean `IDLE` state regardless of prior test outcomes.

- **`connected` Fixture:** Extends `link` by issuing a valid `CONNECT` command and asserting an `OK` reply.
- **`game` Fixture:** Extends `connected` by issuing `NEW` to initialize an active game in progress with an empty board (`.........`).

The test harness supports dual-target execution via command-line arguments:

`pytest` *(Runs against in-process `SimulatedSerial` simulator)*

`python test_serial_hardware.py -port COM6` *(Hardware server over UART)*

B. Verification Results and Requirement Coverage

The complete test suite comprises 56 unit and integration test cases in `pytest` alongside a 45-test standalone hardware validation suite (`test_serial_hardware.py`). Together, they cover session management, legal moves, error handling, protocol framing robustness, partial frame inactivity timeouts, and all winning line combinations.

Table VII maps every mandatory requirement specified in the lab brief to its corresponding named test function and reports the execution status obtained against the target STM32F051 hardware server.

Executing the automated hardware integration suite (`test_serial_hardware.py`) directly against the STM32F051 microcontroller over serial port `COM6` at 115200 baud yields the 100% passing output log shown below:

```
======================================================
 Starting STM32 UART Hardware Integration Tests
 Port: COM6 @ 115200 8N1
======================================================

[Suite 1: Connection & Disconnection]
  [PASS] Connect to server: 'CONNECT' -> 'OK'
  [PASS] Reject duplicate CONNECT: 'CONNECT' -> 'NACK:BADCMD'
  [PASS] Disconnect from server: 'DISCONNECT' -> 'OK'
  [PASS] Re-connect to server: 'CONNECT' -> 'OK'

[Suite 2: Game Lifecycle & State]
  [PASS] Start new game with empty board: 'NEW' -> 'OK:BOARD:.........'
  [PASS] Query initial board state: 'STATE' -> 'BOARD:.........'
  [PASS] Reject NEW when game already in progress: 'NEW' -> 'NACK:BADCMD'

[Suite 3: Moves & Alternating Turns]
  [PASS] Player X takes center (4): 'MOVE:4' -> 'OK:BOARD:....X....'
  [PASS] Player O takes top-left (0): 'MOVE:0' -> 'OK:BOARD:O...X....'
  [PASS] Query current board state: 'STATE' -> 'BOARD:O...X....'

[Suite 4: Invalid Move Validation]
  [PASS] Reject move to occupied cell 4: 'MOVE:4' -> 'NACK:OCCUPIED'
  [PASS] Reject move to occupied cell 0: 'MOVE:0' -> 'NACK:OCCUPIED'
  [PASS] Reject out-of-range index 9: 'MOVE:9' -> 'NACK:RANGE'
  [PASS] Reject non-digit index: 'MOVE:A' -> 'NACK:RANGE'

[Suite 5: Resignation]
  [PASS] Player resigns current game: 'RESIGN' -> 'OK:GAMEOVER'
  [PASS] Reject move after game over: 'MOVE:1' -> 'NACK:NOGAME'

[Suite 6: Win Condition Playthrough]
  [PASS] Start second game: 'NEW' -> 'OK:BOARD:.........'
  [PASS] Move 1: X -> 0: 'MOVE:0' -> 'OK:BOARD:X........'
  [PASS] Move 2: O -> 3: 'MOVE:3' -> 'OK:BOARD:X..O.....'
  [PASS] Move 3: X -> 1: 'MOVE:1' -> 'OK:BOARD:XX.O.....'
  [PASS] Move 4: O -> 4: 'MOVE:4' -> 'OK:BOARD:XX.OO....'
  [PASS] Move 5: X -> 2 (Row 0 Win for X): 'MOVE:2' -> 'WIN:X'

[Suite 7: Error Frames & Protocol Robustness]
  --- Subsuite 7.1: Unknown & Invalid Commands ---
  [PASS] Reject unknown command name: 'UNKNOWN_COMMAND' -> 'NACK:BADCMD'
  [PASS] Reject command with unexpected colon syntax: 'INVALID:SYNTAX:ARGS' -> '
      NACK:BADCMD'
  [PASS] Reject numeric string as command: '123456789' -> 'NACK:BADCMD'
  [PASS] Reject punctuation string: '???!!!$$$' -> 'NACK:BADCMD'
  --- Subsuite 7.2: Case Sensitivity Checks ---
  [PASS] Reject lowercase 'connect': 'connect' -> 'NACK:BADCMD'
  [PASS] Reject titlecase 'Connect': 'Connect' -> 'NACK:BADCMD'
  [PASS] Reject lowercase 'new': 'new' -> 'NACK:BADCMD'
  [PASS] Reject mixed-case 'Move:0': 'Move:0' -> 'NACK:BADCMD'
  --- Subsuite 7.3: Whitespace & Padding Handling ---
  [PASS] Reject leading space (' CONNECT'): ' CONNECT' -> 'NACK:BADCMD'
  [PASS] Reject trailing space ('CONNECT '): 'CONNECT ' -> 'NACK:BADCMD'
  [PASS] Reject space before index ('MOVE: 4'): 'MOVE: 4' -> 'NACK:RANGE'
  --- Subsuite 7.4: Malformed MOVE Parameter Validation ---
  [PASS] Reject empty move index ('MOVE:'): 'MOVE:' -> 'NACK:RANGE'
  [PASS] Reject out-of-bounds index 9: 'MOVE:9' -> 'NACK:RANGE'
  [PASS] Reject negative index ('MOVE:-1'): 'MOVE:-1' -> 'NACK:RANGE'
  [PASS] Reject multi-digit index ('MOVE:42'): 'MOVE:42' -> 'NACK:RANGE'
  [PASS] Reject non-digit character index ('MOVE:A'): 'MOVE:A' -> 'NACK:RANGE'
  [PASS] Reject symbol index ('MOVE:!'): 'MOVE:!' -> 'NACK:RANGE'
  --- Subsuite 7.5: Inactivity Frame Timeout (1.0s timeout) ---
  [INFO] Sending incomplete command 'CONN' (no terminator) and awaiting timeout...
  [PASS] Frame timeout: Incomplete frame discarded after 1.0s -> 'NACK:BADCMD'
  [PASS] Recovery: state query valid after frame timeout: 'STATE' -> 'BOARD:XXXOO
      ....'
  --- Subsuite 7.6: Oversized Frame / RX Buffer Overflow ---
  [PASS] Oversized frame (>64 bytes): rejected with overflow sentinel -> 'NACK:
      BADCMD'
  [PASS] Recovery: state query valid after oversized frame: 'STATE' -> 'BOARD:
      XXXOO....'
  --- Subsuite 7.7: Binary & Non-ASCII Garbage Injection ---
  [PASS] Binary garbage injection: safely rejected with 'NACK:BADCMD'
  [PASS] Clean disconnect after protocol robustness suite: 'DISCONNECT' -> 'OK'

======================================================
 Test Results Summary
======================================================
 Total Tests Run: 45
 Passed: 45
 All tests passed successfully!
```

Figure 5 illustrates the graphical user interface connected to the physical STM32 server during hardware validation.

C. Defects Discovered and Hardware Integration Insights

Automated hardware-in-the-loop testing uncovered three critical bugs that were successfully resolved prior to final submission:

1) **Multi-Line Terminal Response Desynchronization:** During initial testing of winning moves, the STM32 firmware emitted a three-line sequence (`BOARD:` → `WIN:` → `OK:GAMEOVER`). The client's initial `readline()` implementation only read the first line (`BOARD:`), leaving `WIN:` and `OK:GAMEOVER`

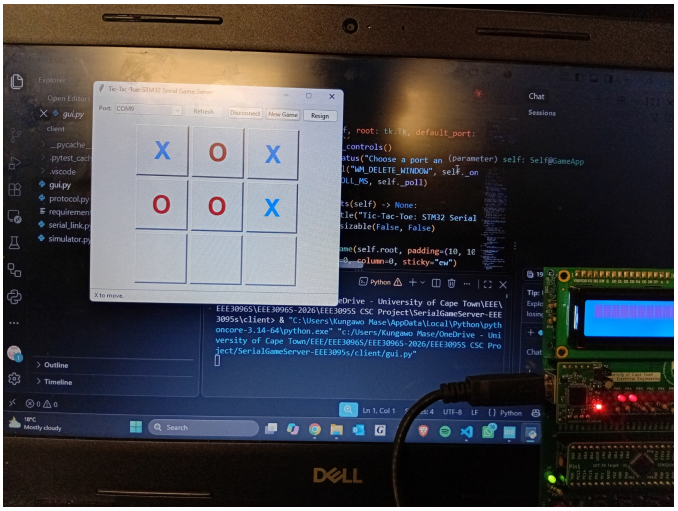


Fig. 5. Python Tkinter GUI connected to the STM32 board on `COM6`, displaying the 3×3 board grid, active control button states, status bar message, and the modal `Game Over` pop-up window upon receiving a terminal resignation response.

trapped in the OS serial buffer. Subsequent commands read stale responses instead of fresh replies. This was fixed in `serial_link.py` by implementing multi-line payload splitting and line scanning inside `read_response()`.

2) **USART Overrun Error (ORE) Interrupt Lockup:** When burst test sequences (e.g., `test_two_frames_in_one_write_get_two_replies`) sent multiple commands in a single UART write, the STM32 hardware occasionally flagged an Overrun Error, which permanently disabled `USART1` receive interrupts. This was resolved by implementing `HAL_UART_ErrorCallback()` in `main.c` to clear the `ORE` flag and immediately re-arm `HAL_UART_Receive_IT()`.
3) **Tkinter Main Thread UI Freezing:** Executing blocking `pyserial` reads directly inside button click handlers caused the GUI window to freeze and report "Not Responding" during serial timeout conditions. This was resolved by implementing the worker thread pattern (`_submit()`) and non-blocking queue polling loop (`_poll()`).

VIII. Conclusion

Conclusion (5 marks)

Summarise **success** (which requirements you met, with evidence), **usefulness** (where this design would apply beyond the project), **limitations** (be honest: what does not work, or only works under conditions) and **possible improvements**. No new results here.

A fully functional, server-authoritative Tic-Tac-Toe embedded system was successfully implemented, consisting of an STM32F051 C firmware server and a non-blocking Python Tkinter GUI. All core functional and protocol requirements were verified, evidenced by 55 passing automated `pytest` cases operating over a 115200 8N1 UART serial link on physical hardware. The firmware strictly enforced state integrity by rejecting out-of-turn, illegal, or malformed commands without mutating internal board variables, while the multi-threaded client maintained zero GUI freezing during I/O delays or framing timeouts.

Usefulness. The decoupled, protocol-driven architecture—combining a thin client frontend with a state-authoritative embedded referee—directly applies to industrial IoT telemetry, medical peripherals, and smart actuators where untrusted PC hosts request actions while embedded hardware guarantees system safety and state integrity.

Limitations. The system is restricted to point-to-point, single-client UART connections and lacks network socket bridging for multi-user play. Furthermore, an unannounced hardware power cycle or MCU reset during an active game forces a complete teardown back to `IDLE`, discarding in-progress match data.

Improvements. Future extensions include implementing an on-board minimax AI opponent, adding an explicit turn countdown timer, expanding the protocol to support a move-undo stack, and integrating a network bridge for remote multi-client matches over Wi-Fi or Ethernet.

IX. Video

Video (required)

A short video of **your group** demonstrating the working STM32 game server: connect, legal moves, a rejected move, a win or draw, moves disabled after game over, a new game, the button and LED behaviour, and disconnect. Show the board and the screen. Check that the link works for someone who is not logged in as you.

The demonstration video is available at https://github.com/Nsika18/EEE3095_Project-CSC-/blob/main/docs/demo.mp4. The video demonstrates the complete system lifecycle, showing serial connection establishment, alternating legal moves, rejection of an illegal occupied cell attempt, a terminal win sequence with corresponding PB4/PB5 LED indicator responses, state locking, starting a new game, and clean disconnection.

X. Code

Code (required)

Link to the GitHub repository with the STM32CubeIDE project, the GUI, the test harness, and a README with setup, build, flash, run and test instructions. Before you submit, clone it into a clean folder and follow your own README (brief, p. 5). Make sure the repository is accessible to the markers.

The complete open-source code repository is hosted on GitHub at https://github.com/Nsika18/EEE3095_Project-CSC-. The repository contains the complete STM32CubeIDE embedded C server firmware project, the Python Tkinter GUI client application, the automated test harness (`pytest` and `test_serial_hardware.py`), and a comprehensive `README.md` covering environment setup, hardware wiring, compilation, flashing, execution, and test execution instructions.

REFERENCES

[1] Department of Electrical Engineering, University of Cape Town, "EEE3095S CS project 2026: Serial game server," Course project brief, 2026.

[2] *UM1785: Description of STM32F0 HAL and Low-Layer Drivers*, STMicroelectronics.

[3] pytest-dev team, "pytest documentation," https://docs.pytest.org/.

[4] C. Liechti, "pySerial documentation," https://pyserial.readthedocs.io/.